

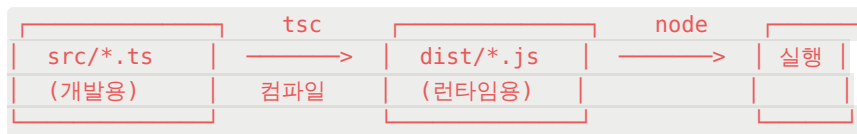
ESM

📅 날짜	2026년 4월 21일
☀️ 상태	시작 전

ESM 규칙: "import 경로는 실행 파일 기준"

핵심 사실: TypeScript는 그대로 안 돌아간다

Node.js는 `.ts` 파일을 직접 실행하지 못합니다. 반드시 `.js` 로 컴파일한 결과물을 실행합니다.



→ **실제로 돌아가는 파일은 `dist/*.js` **입니다.

import 경로는 누가 보나?

Node.js(런타임)가 봅니다. 그리고 Node.js는 `dist/` 폴더에서 동작하니까, 거기 기준으로 경로를 해석합니다.

```
// src/board/sprints/sprints.module.ts (개발 시 우리가 보는 파일)
import { SprintsService } from './sprints.service.js';
// ~~~~~
// 런타임엔 dist/board/sprints/sprints.service.js를 가리킴
```

→ 컴파일 후엔 `.ts` 가 사라지고 `.js` 만 남으니, import 경로도 `.js` 로 써야 함.

TypeScript는 왜 자동으로 안 바꿔주나?

TS의 철학: "타입만 추가하고 JS 코드는 손대지 않는다."

`import` 문은 JS 코드의 일부라서, TS가 컴파일할 때 그대로 둡니다. 즉:

```
// 우리가 쓴 것 (src/main.ts)
import { AppModule } from './app.module.js';

// 컴파일 결과 (dist/main.js)
import { AppModule } from './app.module.js'; // ← 그대로
```

만약 `.ts` 로 쓰면 컴파일 후에도 `.ts` 로 남아서 → Node.js가 못 찾고 에러.

비교: CommonJS는 다르다

```
// CommonJS (Node.js 전통 방식)
const x = require('./sprints.service'); // ← 확장자 생략 OK
```

CommonJS는 Node.js가 자체 해석 로직(확장자 자동 추측)을 갖고 있어서 생략 가능.
ESM은 표준이라 그런 관용을 안 봐줌 — 정확한 경로 필수.

한 문장 요약

"실행될 때 존재할 파일을 적어야 한다. **.ts** 는 컴파일 후 **.js** 가 되니, **import**는 **.js** 로."

이질감이 들더라도 **이 한 가지** 규칙만 기억하면 됩니다. 다음 파일로 넘어갈까요?